QR-Based Restaurant Ordering Web Application

Technical Architecture, QA Audit, Feature Documentation, and Free Deployment Guide

Field	Details
Project	restaurant-ordering-app / Qrave
Prepared for	Client, development team, and potential stakeholders
Prepared by	Senior full-stack engineering and product QA review
Repository	C:\Users\ashok\Documents\FreeLancing Projects\Sample App\Resturant-app
Document purpose	Explain how the system works, what is implemented, what still needs attention, and how to deploy it affordably.

Executive note

The application is demo-ready and suitable for stakeholder walkthroughs. For live restaurant operations, the main remaining requirements are persistent production database configuration, secure admin credentials, stronger testing, and operational monitoring.

Table of Contents

Table of Contents 2
1. Project Overview 3
2. System Architecture 3
Folder Structure 3
3. Features Breakdown 4
Customer App 4
Admin and Kitchen Panel 4
QR Generator 4
4. Workflow Explanation 4
5. Code Quality Review 4
Positive Practices 5
Anti-Patterns and Risks 5
6. Bugs and Issues Found 5
7. Improvements and Recommendations 6
UI and UX 6
Backend and Operations 6
SaaS Scalability 6
8. Free Deployment Guide 6
Frontend Deployment on Vercel 6
Frontend Deployment on Netlify 6
Backend Deployment on Render 7
Backend Deployment on Railway 7
Domain Setup 7
QR Code Update After Deployment 7
Common Deployment Errors 7
9. Deployment Readiness Check 8
10. Future Roadmap 8
Appendix A: Verified Implementation Notes 9
Appendix B: Regenerating This Document 9

1. Project Overview

Qrave is a QR-based restaurant ordering web application. Customers scan a table QR code, open a table-specific menu route, browse menu categories, add items to a cart, and place an order. Restaurant staff use an admin/kitchen dashboard to view live orders, update preparation status, manage menu items, and generate printable table QR codes.

Area	Description
Project name	restaurant-ordering-app, branded in the UI as Qrave.
Purpose	Reduce waiter dependency for order capture and provide a self-service table ordering workflow.
Target users	Restaurant customers, kitchen staff, admins, restaurant owners, and demo stakeholders.
Core concept	Each table has a QR code that opens /menu?table=N. The table number is captured and included with every order.

2. System Architecture

The project is structured as a React client and an Express API server. It supports two operating modes: a server-backed mode using MongoDB and Socket.io, and a local demo fallback mode using localStorage when the API is unavailable.

Layer	Technology	Role
Frontend	React 18, Vite, React Router, Tailwind CSS, Framer Motion, Lucide Icons	Customer menu, cart, checkout, order status, admin dashboard, QR generator, theme toggle.
API server	Express, Mongoose, Socket.io, JWT, Helmet, CORS, Compression	Menu APIs, order APIs, auth, real-time order events, demo store fallback.
Database	MongoDB via Mongoose	Persistent menu, order, and admin data in production.
Fallback data	Browser localStorage and in-memory server demo store	Static/demo operation when backend or MongoDB is not available.
QR generation	qrcode npm package	Creates PNG data URLs for table-specific menu links.

Folder Structure

Path	Purpose
client/src/pages	Top-level React screens: CustomerMenu, OrderStatus, AdminLogin, AdminDashboard.
client/src/components	Reusable UI components including cart bar, menu cards, checkout modal, QR generator, navigation, and status stepper.
client/src/services	API client, Socket.io client, and localStorage fallback store.
server/src/controllers	Business logic for auth, menu, orders, and payments.
server/src/models	Mongoose schemas for Admin, Menu, and Order.
server/src/routes	Express route definitions for API endpoints.
server/src/data	Seeded demo menu and demo order store.

3. Features Breakdown

Customer App

- QR routing is implemented through /menu?table=N, with legacy /table/:tableNo still supported.
- Table number is normalized and saved to localStorage and sessionStorage.
- Menu data loads from /api/menu when available and falls back to local demo menu data.
- Categories are derived from menu data and rendered as scrollable tabs.
- Cart supports add, increment, decrement, remove-by-zero, total calculation, and per-table localStorage persistence.
- Checkout validates non-empty cart and customer name before placing an order.
- Order success uses a toast notification with the exact message 'Order placed successfully'.
- Order status page shows table, customer, line items, total, and status progression.

Admin and Kitchen Panel

- Admin login exists with JWT-backed API auth and a local demo fallback for default credentials.
- Dashboard displays order count, revenue, in-kitchen count, served count, and an order-flow chart.
- Live orders table shows customer, table, items, total, time, and status buttons.
- Status flow is pending -> preparing -> served.
- Socket.io is used for server-backed real-time order notifications and updates.
- Menu manager supports creating, editing, deleting, and availability control for menu items.
- A short sound notification is attempted for new server-pushed orders, subject to browser autoplay rules.

QR Generator

- Admin dashboard includes a 'Generate table QR codes' section.
- Input fields support number of tables and base URL, defaulting to window.location.origin.
- For each table, the generated URL follows the format {baseUrl}/menu?table=N.
- Each table QR displays a table label, 'Scan to Order' text, the encoded URL, and a PNG download button.
- Print support is implemented through a print-optimized grid layout.
- A 'Scan Me Preview' section shows the first generated table QR code.

4. Workflow Explanation

1. Restaurant staff generate QR codes in the admin dashboard and print one code per table.

2. A customer scans a table QR. The browser opens /menu?table=N, where N is the assigned table number.

3. The customer browses categories, adds items to cart, changes quantities, and reviews the total.

4. The checkout modal collects the customer name and submits the order with table number and line items.

5. The backend validates menu item availability, calculates total, prevents very recent duplicate submissions, stores the order, and emits a Socket.io event.

6. The admin dashboard receives the order and staff move it through pending, preparing, and served.

7. The customer can follow the order status page for updates.

5. Code Quality Review

Category	Assessment
Structure	Clear client/server split with pages, components, services, controllers, routes, and models.
Modularity	Good reuse of UI components and service wrappers. QR generation and local store are separated cleanly.

Category	Assessment
Naming	Mostly readable names. One minor consistency issue remains: tableNo and table both exist in fallback order shape.
Maintainability score	7.8 / 10 for a demo-ready app. It is organized and understandable, but needs tests, stronger production auth, and better backend operational hardening.
Developer experience	Root scripts are present. Vercel and Netlify SPA rewrite files are included.

Positive Practices

- API calls are centralized through an Axios service.
- Socket logic is isolated from page components.
- The UI uses reusable components and consistent Tailwind utility patterns.
- Server controllers validate important inputs such as empty orders and invalid statuses.
- Deployment routing is handled through platform-specific rewrite files.

Anti-Patterns and Risks

- No automated test suite is present for customer flows, admin flows, API contracts, or QR generation.
- Default admin credentials are demo-friendly but unsafe for production if not changed.
- The localStorage fallback is excellent for demos but not a multi-device source of truth.
- Some image assets depend on external URLs, so menu visuals rely on third-party availability.
- Admin destructive actions, such as menu deletion, do not require confirmation.
- Payment integration exists as server code but is not presented as a completed customer checkout flow.

6. Bugs and Issues Found

Severity	Issue	Impact	Recommendation
High	Production admin credentials must be changed.	Unauthorized access risk if demo defaults are reused.	Set strong credentials and protect admin creation/reset workflow.
High	No automated tests.	Regression risk in order placement, QR routing, and admin status updates.	Add Playwright end-to-end tests and API tests.
Medium	localStorage fallback is not shared across devices.	Static-only deployment cannot support real kitchen devices seeing customer orders.	Use the backend with MongoDB or Firebase for live operations.
Medium	Menu delete has no confirmation dialog.	Accidental data loss in admin panel.	Add confirm modal or soft-delete status.
Medium	External image URLs can fail or slow the menu.	Broken visuals and inconsistent performance.	Upload optimized restaurant-owned images or use a CDN.
Medium	Socket URL and API URL must be configured per environment.	Admin real-time updates fail if env variables point to the wrong host.	Set VITE_API_URL, VITE_SOCKET_URL, CLIENT_URL, and CORS allowlist correctly.
Low	QRCode 'download all' is implemented as print grid, not zip.	Acceptable for operations, but not a true bulk PNG export.	Add JSZip if bulk downloadable PNG files are required.
Low	Sound notification can be blocked until user interaction.	New order sound may not play immediately in some browsers.	Show visual notification and request interaction/permission where appropriate.

7. Improvements and Recommendations

UI and UX

- Add restaurant branding controls for logo, colors, opening hours, and service notices.
- Add item modifiers such as spice level, add-ons, notes, and variants.
- Add a clearer empty-cart state and optional order review screen before final placement.
- Add table-specific service actions such as 'Call waiter' and 'Request bill'.

Backend and Operations

- Move from demo data to managed MongoDB Atlas for production.
- Add rate limiting, request validation schemas, and structured server logging.
- Add admin role permissions for owner, manager, and kitchen staff.
- Add audit logs for status updates and menu changes.
- Add backup/export capability for orders and menu data.

SaaS Scalability

- Introduce tenant/restaurant IDs across menus, orders, admins, and QR codes.
- Support multiple branches, tables per branch, subscription plans, and branded customer URLs.
- Add analytics for average order value, peak hours, best-selling items, kitchen time, and repeat customers.
- Separate public customer API from internal admin API and enforce stricter access controls.

8. Free Deployment Guide

Beginner-friendly summary

Deploy the frontend to Vercel or Netlify for free. Deploy the backend to Render or Railway free tier. Use MongoDB Atlas free tier for the database. After deployment, regenerate QR codes using the live frontend URL.

Frontend Deployment on Vercel

1. Create a GitHub account and push the project repository to GitHub.

2. Go to vercel.com and sign in with GitHub.

3. Click Add New Project and import the repository.

4. Set the Root Directory to client.

5. Use npm install as the install command and npm run build as the build command.

6. Use dist as the output directory.

7. Add environment variables when using a backend: VITE_API_URL=https://your-api.onrender.com/api and VITE_SOCKET_URL=https://your-api.onrender.com.

8. Click Deploy. Vercel provides a free .vercel.app URL.

Frontend Deployment on Netlify

1. Push the repository to GitHub.

2. Go to netlify.com and create a new site from Git.

3. Select the repository and set Base directory to client.

4. Set Build command to npm run build and Publish directory to client/dist.

5. Add the same VITE_API_URL and VITE_SOCKET_URL environment variables if the backend is deployed.

6. Deploy the site. The included _redirects file supports React Router refreshes.

Backend Deployment on Render

1. Create a free MongoDB Atlas cluster and copy the connection string.

2. Go to render.com and create a new Web Service from the GitHub repository.

3. Set Root Directory to server.

4. Set Build Command to npm install.

5. Set Start Command to npm start.

6. Add environment variables: MONGO_URI, JWT_SECRET, CLIENT_URL, PORT if needed, and ALLOW_DEMO_STORE=false for production.

7. Deploy and copy the backend URL, for example https://your-api.onrender.com.

8. Return to Vercel or Netlify and set VITE_API_URL and VITE_SOCKET_URL using the backend URL.

Backend Deployment on Railway

- Railway can deploy the server directory from GitHub and provide environment variables through its dashboard.
- Use the same required variables: MONGO_URI, JWT_SECRET, CLIENT_URL, and optionally ALLOW_DEMO_STORE=false.
- Free availability can change over time, so confirm the current free limits before using it for a client.

Domain Setup

- Free subdomains are included by Vercel, Netlify, Render, and Railway.
- For a custom domain, buy a domain or use a free/low-cost provider, then add the domain inside Vercel or Netlify.
- Update DNS records exactly as the hosting platform instructs.
- After the custom domain works, regenerate all QR codes with the new public URL.

QR Code Update After Deployment

1. Open the deployed admin dashboard.

2. Go to Generate table QR codes.

3. Set Base URL to the live frontend domain, not localhost.

4. Enter the number of tables.

5. Download or print the generated table QR codes.

6. Physically test at least one QR code for table 1, one middle table, and the last table.

Common Deployment Errors

Problem	Likely Cause	Fix
Images not loading	External URL blocked, removed, or slow.	Use reliable CDN-hosted images or upload restaurant-owned images.
API not working	Wrong VITE_API_URL or CORS CLIENT_URL missing.	Check frontend env vars and backend allowed origins.
Admin live updates not	Wrong VITE_SOCKET_URL or	Set socket origin to backend domain and confirm

Problem	Likely Cause	Fix
working	WebSocket blocked.	Render/Railway supports WebSockets.
Refresh gives 404	SPA rewrite missing.	Use client/vercel.json on Vercel or client/public/_redirects on Netlify.
Orders disappear	Running only static localStorage demo mode.	Deploy backend and MongoDB for real multi-device persistence.
Mongo connection fails	Incorrect MONGO_URI or IP/network restrictions.	Use MongoDB Atlas connection string and correct database user password.

9. Deployment Readiness Check

Area	Current Status	Before Live Use
Customer ordering	Demo-ready and functional.	Run real-device QA across common mobile browsers.
Admin dashboard	Demo-ready with live order management.	Secure credentials and add role-based access.
QR generator	Implemented with PNG and print workflow.	Regenerate using deployed URL and test printed codes.
Persistence	MongoDB supported; local fallback exists.	Configure production MongoDB Atlas and disable demo fallback.
Security	Basic JWT auth and Helmet are present.	Change defaults, add rate limits, validate all inputs, and protect admin operations.
Testing	Manual QA performed; no automated tests found.	Add automated regression tests before live restaurant rollout.

Readiness verdict

The app is ready for demos, investor walkthroughs, and controlled pilots. It should not be treated as fully production-hardened until database, credentials, monitoring, backups, validation, and automated tests are completed.

10. Future Roadmap

Phase	Roadmap Items
Phase 1: Live pilot	Production MongoDB, secure admin, real restaurant menu, printed QR codes, staff training, mobile QA.
Phase 2: Operational polish	Order notes, item modifiers, waiter calls, bill request, kitchen filters, order timers, receipt export.
Phase 3: Payments	Razorpay or Stripe checkout, payment status, refunds, settlement reporting.
Phase 4: SaaS	Multi-restaurant tenancy, branch management, subscriptions, owner analytics, branded domains.
Phase 5: Advanced analytics	Sales dashboards, item performance, table turnover, peak hour predictions, inventory signals.

Appendix A: Verified Implementation Notes

- Browser QA verified /menu?table=3 table detection, cart persistence after refresh, order placement, admin order visibility, QR code URL generation, /menu?table=7 routing, and no observed console errors during those flows.
- Production client build completed successfully with npm run build --prefix client.
- Server syntax checks passed for the modified order controller and Order model.
- The app currently running in the in-app browser is at http://127.0.0.1:5173/admin/dashboard.

Appendix B: Regenerating This Document

From the repository root, run:

```
"C:\Users\ashok\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe"
tools\generate_restaurant_app_documentation.py
```